Software Design Specification

Atlas-Indexed Reversible Evidence-Fibered Geodesic πPLN

A Wrapper-First Implementation on patham9 PLN with Native Geodesic-Control Interoperability

Ben Goertzel
with GPT-5.6 Pro assistance

July 12, 2026

Abstract

This document specifies an implementable software architecture for a broad form of Probabilistic Logic Networks in which persistent knowledge is a context-indexed, provenance-rich, paraconsistent evidence metagraph; scalar PLN truth values are temporary projections inside local probabilistic charts; proof search is guided by bidirectional geodesic inference control; repeated proof paths do not duplicate evidence; context changes and revisions are explicit; and inference execution is replayable and reversibly auditable. The implementation target is the public patham9 PLN codebase, retained with as few changes as possible.

The central boundary is strict. patham9 remains a compact local scalar-STV derivation kernel. The surrounding πPLN runtime owns contexts, charts, positive and negative evidence packets, provenance, exact or conservative overlap handling, geodesic control policy, proof identity, reversible event history, curvature diagnostics, chart gluing, revision, and promotion. A patham9 invocation is therefore a temporary local inference episode rather than the persistent epistemic state.

The design supports three kernel-control modes behind one capability-negotiated interface: the current public patham9 deriver operated in externally controlled microsteps; an optional overlay deriver exposing policy and event hooks without editing the vendor tree; and a future native patham9 geodesic inference-control backend. The outer atlas controller always retains authority over context selection, evidence snapshots, chart assumptions, cross-context translations, revision, promotion, and global resource budgets. A native patham9 geodesic controller, when available, owns only within-chart proof-path scheduling under an explicit control envelope.

The specification defines normative invariants, content-addressed data records, AtomSpace annotations, Python protocols, MeTTa overlays, episode workflows, exact stamp mapping, projection policies, proof-class de-duplication, reversible execution records, sparse commutator probes, holonomy and descent diagnostics, persistence and concurrency rules, security gates, test properties, migration from the existing partial wrapper, staged work packages, and an appendix giving operational instructions to coding agents. The result is intended to be both a system design and a contract: implementation choices may vary, but evidence conservation, chart locality, context typing, auditability, deterministic replay, and separation of control mass from evidence mass are non-negotiable.

Contents

1 Document purpose, status, and normative language

1.1 Purpose

This specification answers the engineering question:

> How can an atlas-indexed, reversible, evidence-fibered, geodesically controlled πPLN be implemented on top of patham9 PLN while preserving the working patham9 kernel and remaining ready to consume a forthcoming native geodesic inference-control implementation?

The intended readers are implementers of petta-memory, Hyperon/MeTTa integration code, patham9 adapters, proof and evidence stores, inference controllers, evaluation harnesses, and autonomous coding agents assigned to implement bounded work packages.

1.2 Status

This is a standalone design specification. It distinguishes:

1. behavior already present in the inspected wrapper effort;
2. behavior available from the current public patham9 kernel;
3. behavior specified for the first production implementation;
4. optional advanced behavior whose mathematics is defined but whose implementation is staged;
5. integration seams for a future patham9-native geodesic controller whose public API is not assumed here.

The public patham9 baseline inspected for this document consists of the MeTTa modules `Utils.metta`, `Constraints.metta`, `Formulas.metta`, `Rules.metta`, `Config.metta`, and `Deriver.metta`, concatenated by the repository build script into `PLN.metta`; `Translator.metta` is a separate translation utility [1, 2, 3, 4, 5]. The current partial πPLN effort is a wrapper-first, read-only bridge with contextual evidence packets, numeric stamps plus a provenance sidecar, projection and context filters, bounded inference-control plans, and a fail-closed runtime gate [7].

1.3 Normative terms

The terms MUST, MUST NOT, REQUIRED, SHOULD, SHOULD NOT, and MAY are normative.

- MUST identifies a property required for semantic or safety correctness.
- SHOULD identifies the preferred design; alternatives require an architectural decision record and equivalent tests.
- MAY identifies an optional capability.

2 Goals, non-goals, and success criteria

2.1 Primary goals

The system MUST:

G1. preserve patham9 as a small, replaceable local derivation kernel;

G2. make persistent evidence context-indexed, positive/negative, provenance-rich, and correctable;

G3. treat priors and scalar STVs as chart-local projection data rather than persistent empirical truth;

G4. prevent known evidence reuse from inflating confidence;

G5. keep syntactically distinct proofs for search and explanation while merging evidence idempotently when they are evidence-equivalent;

G6. support forward, backward, and bidirectional geodesic control without conflating controller utility with PLN confidence;

G7. support future native patham9 geodesic control without requiring the outer architecture to be rewritten;

G8. isolate incompatible charts and require explicit translations or covering contexts for cross-context aggregation;

G9. provide deterministic replay and reversible audit for every accepted derivation and revision;

G10. keep proof-order curvature, chart non-gluability, conflict, and evidence multiplicity as separate diagnostics;

G11. remain read-only by default and require an explicit typed promotion gate for durable belief or evidence changes;

G12. provide precise tests and coding-agent constraints that make semantic regressions difficult to introduce accidentally.

2.2 Non-goals for the initial implementation

The initial implementation MUST NOT claim to provide:

N1. native positive/negative evidence semantics inside every patham9 truth formula;

N2. a universal inverse from an arbitrary derived STV to empirical evidence counts;

N3. one global probability distribution underlying all contexts;

N4. exact causal independence merely from distinct source identifiers;

N5. a fully mechanized directed PGM–HoTT type theory;

N6. exact differential-geometric curvature for arbitrary symbolic rules;

N7. automatic persistent promotion of inferred statements;

N8. a commitment to the concrete API of any unreleased patham9 geodesic controller.

2.3 System-level success criteria

A release is successful when it demonstrates all of the following:

1. stock patham9 examples and rule tests remain unchanged in compatibility mode;
2. repeated construction of the same chart produces no persistent evidence increment;
3. two proof paths with the same primitive provenance do not increase persistent confidence beyond one path;
4. two independent provenance sets can combine;
5. partially overlapping evidence is rejected or explicitly residualized, never silently added;
6. the same surface term in incompatible contexts remains separate without a translation witness;
7. a complete run can be replayed from immutable inputs, configuration, seed, and kernel version;
8. an accepted revision can be undone by moving the active pointer to the prior version;
9. controller decisions are reproducible and separately auditable from object-level inference;
10. the runtime can switch between current microstep control and a simulated native geodesic backend through one interface;
11. a known non-gluable chart family returns a non-gluability object rather than a forced global scalar;
12. no ordinary inferential rule creates a primitive evidence token.

3 Reference kernel analysis and design constraints

3.1 Current patham9 sentence contract

The current deriver pattern-matches sentences of the form

```
(Sentence ($Term (stv $Strength $Confidence)) $Stamp)
```

The implementation MUST NOT add positional fields to this shape. Rich metadata MUST be stored in separate annotations or sidecar records.

3.2 Current scheduling behavior

The public deriver defines

$$\texttt{PriorityRank(Sentence)} = c, \tag{3.1}$$

selects the highest-confidence task, applies unary and pairwise rules, places results back into bounded task and belief queues, and recurses until the step budget is exhausted [2]. Consequently:

Invariant 3.1 (Semantic confidence is not controller utility)**.** *A geodesic or task-control score MUST NOT be encoded by overwriting the STV confidence passed to patham9. Confidence is consumed by truth formulas and by the legacy queue rank. Controller utility MUST remain a separate field.*

3.3 Current stamp behavior

The deriver accepts a binary rule application only when `StampDisjoint` finds no equal stamp element. It concatenates and insertion-sorts the premise stamps before attaching their union to the conclusion [2]. Since insertion sort uses the numeric comparison operator, the safest compatibility representation is an episode-local list of collision-free integers.

Invariant 3.2 (Stamp semantics)**.** *Every stamp integer MUST denote one immutable evidence-basis unit within exactly one inference episode. Equal integers MUST mean evidence overlap. Unequal integers MUST NOT be treated as proof of causal independence outside the declared evidence-basis policy.*

3.4 Current truth formulas

The stock rule set includes revision, modus ponens, deduction, induction, abduction, inversion, negation, equivalence conversion, transitive similarity, evaluation implication, and member deduction. The revision formula converts confidence to weight using

$$w = \frac{c}{1-c} \tag{3.2}$$

and combines scalar strengths by a weighted average [3, 4]. This is useful within a compatible scalar chart, but it does not preserve all πPLN evidence distinctions.

Invariant 3.3 (Empirical aggregation precedes prior-aware projection)**.** *Empirical packets that belong to one statement and chart MUST be aggregated with provenance handling before a prior-aware STV is produced. Independently prior-smoothed STVs MUST NOT be fed into stock revision as if the chart prior were independent evidence in each input.*

3.5 Translator isolation

The translator constructs term-level `STV` functions and rule functions from input implications [5]. Incompatible charts MAY assign different STVs to the same term. Therefore each charted episode SHOULD run in a fresh MeTTa space or isolated process, and compiled definitions MUST be keyed by a chart fingerprint.

3.6 Implication for the architecture

The patham9 kernel is best treated as a pure or nearly pure macro-step function:

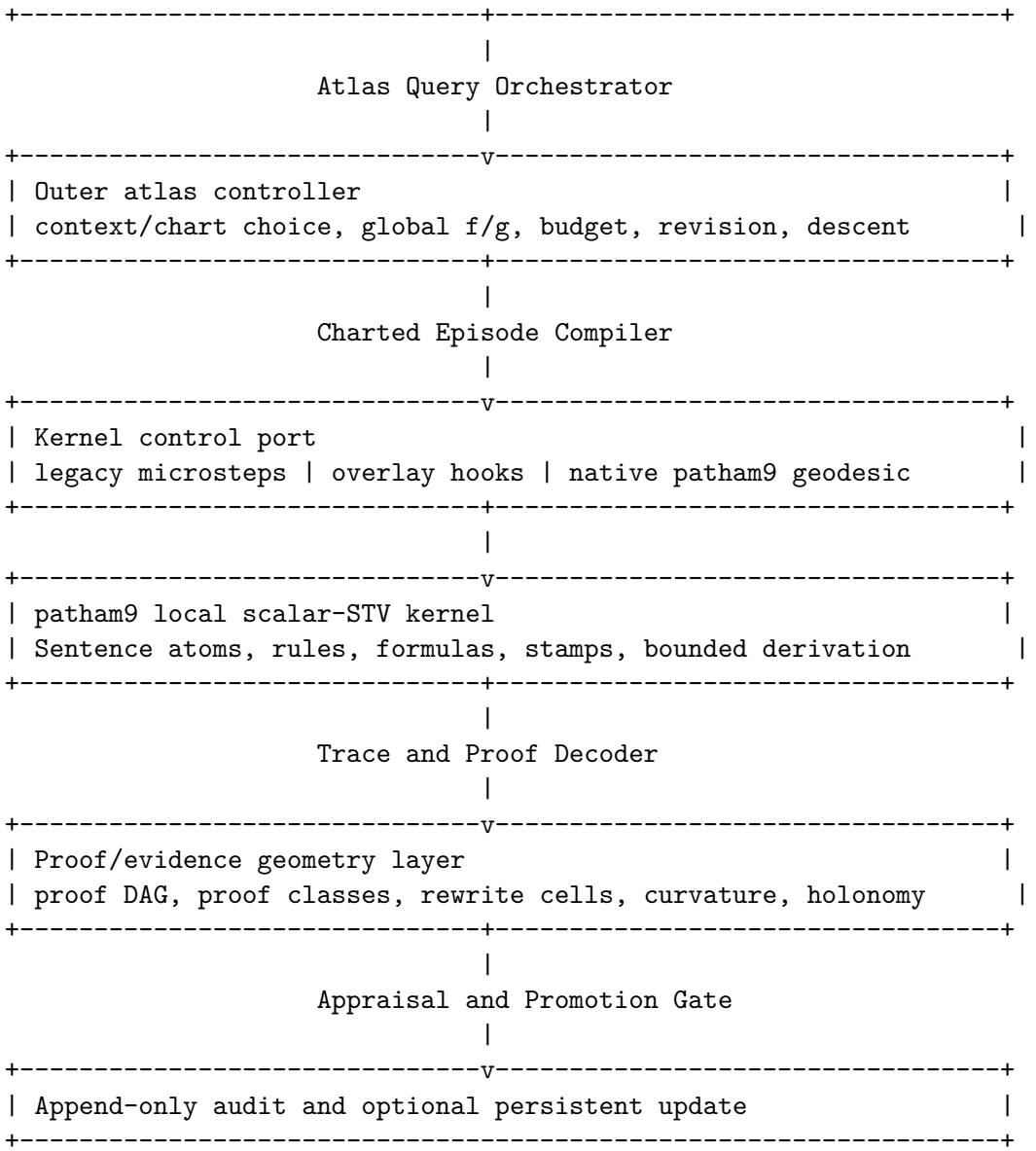
$$K_{v,r} : (\text{sentences}, \text{budget}, \text{seed}) \longrightarrow (\text{sentences}, \text{trace}), \tag{3.3}$$

where v is the pinned kernel version and r is a rule profile. Everything not represented by the sentence/STV/stamp triple lives outside K.

4 Architectural overview

4.1 Layered architecture

```
+-------------------------------------------------------------+
| Persistent evidence and context layer                       |
| packets, contexts, charts, assumptions, translations, revisions  |
```



4.2 Five logical spaces

A Hyperon/AtomSpace realization SHOULD expose five named logical spaces, even if the physical store is shared:

Space	Responsibility
`&pi-evidence`	Primitive evidence tokens, empirical packets, active/quarantine status, reliability, time, causal grouping, exact provenance.
`&pi-atlas`	Contexts, guards, charts, priors, assumptions, context maps, covering contexts, chart compatibility and descent attempts.
`&pi-proof`	Derived assessments, rule applications, proof paths, evidence-aware proof classes, rewrite cells, splices, proof explanations.
`&pi-control`	Forward/backward values, budgets, candidate scores, admission plans, controller decisions, native-kernel control events.

`&pi-audit`	Immutable episode manifests, configuration, seeds, kernel fingerprints, reversible complements, revision and promotion records.

The ephemeral kernel space is created per episode and is not itself a persistent source of truth.

4.3 Authority boundaries

Component	Exclusive authority
Persistent evidence store	Whether a primitive observation exists, its provenance, context, reliability, and active status.
Outer atlas controller	Which contexts and charts are considered, cross-chart moves, global budget allocation, revision actions, and promotion eligibility.
Inner kernel controller	Within one chart, which local rule/path to expand next under an envelope supplied by the outer controller.
patham9 formulas	Local scalar truth-value calculation for an admitted rule instance under the selected rule profile.
Proof geometry layer	Whether two raw paths share evidence, whether a rewrite is coherent, and what order defect or splice mismatch is observed.
Promotion gate	Whether an ephemeral result may become a persistent packet or belief artifact.

5 Architectural decisions and invariants

Architectural Decision 5.1 (Wrapper-first vendor boundary)**.** *The patham9 vendor tree is read-only by default. Extensions are implemented as outer services, generated inputs, separate MeTTa overlays, and adapters. A vendor modification requires a separate decision record showing that the desired property cannot be obtained at the boundary with acceptable complexity or performance.*

Architectural Decision 5.2 (One chart per kernel episode)**.** *A kernel episode has exactly one context, one chart, one assumption fingerprint, one projection policy, one rule profile, and one frozen evidence snapshot. Cross-context inference is represented as several episodes joined by explicit context-transition actions.*

Architectural Decision 5.3 (Packet semantics are primary)**.** *The primitive persistent object is a provenance-tagged evidence packet. Aggregate counts, STVs, beta distributions, controller scores, and proof confidence are derived views.*

Architectural Decision 5.4 (Control/evidence separation)**.** *Raw proof multiplicity may increase control mass, robustness, or explanation diversity. It MUST NOT increase empirical evidence mass unless a path introduces provenance-disjoint primitive evidence.*

Architectural Decision 5.5 (Event-sourced reversibility)**.** *Every accepted kernel transition, chart transition, revision, and promotion is represented by an immutable event containing all inputs, outputs, policies, versions, and complements needed for replay or active-pointer rollback.*

Architectural Decision 5.6 (Capability-negotiated kernel control)**.** *No outer component depends directly on the internal API of the forthcoming patham9 geodesic controller. All kernel scheduling is accessed through the versioned* `KernelControlPort` *defined in section 12.*

Invariant 5.7 (Evidence conservation)**.** *A pure inferential transition MUST satisfy*

$$\mathrm{supp}(T_r e) \subseteq \mathrm{supp}(e). \tag{5.1}$$

Only a typed observation or sampling operation may mint a new primitive evidence token.

Invariant 5.8 (Idempotent fusion)**.** *For every evidence capsule e,*

$$e \sqcup e = e. \tag{5.2}$$

Retry, replay, replication, or duplicate promotion MUST NOT change the active empirical count.

Invariant 5.9 (Context locality)**.** *Two packets or derived assessments with different context fingerprints MUST remain separate unless an explicit covering context and translation witnesses type their comparison.*

Invariant 5.10 (Prior locality)**.** *A chart prior and its virtual counts MUST NOT be stored as empirical evidence. Export from a prior-aware chart MUST either unwind the prior under a typed export policy or remain an ephemeral assessment.*

Invariant 5.11 (Fail-closed promotion)**.** *Absence of a recognized export policy, exact provenance, overlap result, or replayable proof MUST cause promotion rejection.*

6 Identifiers, fingerprints, and versioning

6.1 Identifier classes

All durable records SHOULD have content-addressed identifiers or UUIDs plus a content hash. The following identifiers are distinct types and MUST NOT be interchanged as untyped strings:

Identifier	Meaning
`EvidenceTokenID`	One primitive evidence-basis unit.
`EvidencePacketID`	One signed positive/negative contribution with provenance and context.
`ContextID`	One semantic context and guard definition.
`ChartID`	One local probabilistic chart, including prior and assumptions.
`SnapshotID`	A frozen set of active packets and versions.
`ProjectionID`	One application of a versioned projection policy.
`EpisodeID`	One isolated kernel invocation or native-control session.
`KernelTransitionID`	One selected local derivation transition.
`ProofPathID`	One ordered sequence of transitions.
`ProofClassID`	One evidence-aware equivalence component.
`RewriteCellID`	One explicit comparison of parallel proof fragments.
`RevisionID`	One immutable corrective transformation.
`PromotionID`	One reviewed persistent update decision.
`PolicyID`	A versioned projection, control, overlap, metric, or promotion policy.

6.2 Chart fingerprint

A `ChartFingerprint` MUST hash at least:

```
context_id
context_guard_version
ontology_version
assumption_package_id
prior_strength_p0
prior_weight_k
factorization_policy_id
projection_policy_id
rule_profile_id
evidence_snapshot_id
kernel_version
translator_version
```

No compiled kernel artifact may be reused across unequal chart fingerprints.

6.3 Episode fingerprint

An `EpisodeFingerprint` additionally includes:

```
compiled_sentence_digest
stamp_map_digest
goal_digest
controller_envelope_digest
max_steps
task_queue_size
belief_queue_size
random_seed
kernel_capabilities_digest
```

7 Persistent data model

7.1 Evidence token

```
EvidenceToken {
  id: EvidenceTokenID
  namespace: str
  source_id: str
  source_event_id: str | null
  observed_at: timestamp
  minted_at: timestamp
  context_id: ContextID
  causal_group_id: str | null
  privacy_label: str
  payload_cid: str | null
  schema_version: int
}
```

A token is immutable. `causal_group_id` MAY identify records believed to be manifestations of one causal innovation. Distinct tokens with the same causal group are not automatically independent.

7.2 Evidence packet

```
EvidencePacket {
  id: EvidencePacketID
  statement: CanonicalTerm
  context_id: ContextID
  positive_delta: float
  negative_delta: float
  token_ids: sorted[EvidenceTokenID]
  provenance_digest: str
  source_reliability: float
  temporal_relevance: float
  status: ACTIVE | QUARANTINED | RETRACTED
  assumption_fingerprint: str
  ontology_fingerprint: str
  created_by: OBSERVATION | IMPORT | REVIEWED_EXPORT
  parent_packet_ids: sorted[EvidencePacketID]
  schema_version: int
}
```

The packet may represent a batch, but its token list must expose the overlap granularity required by the active policy. If one token represents a batch, the system treats that batch as an inseparable evidence unit until it is refined.

7.3 Context

```
PiContext {
  id: ContextID
  language_fragment_id: str
  universe_id: str
  guard: GuardExpression
  task_class: str
  assumption_package_id: str
  ontology_id: str
  weakness_policy_id: PolicyID
  relevance_policy_id: PolicyID
  parent_context_ids: sorted[ContextID]
  created_by_revision_id: RevisionID | null
  schema_version: int
}
```

A context is semantic, not merely a tag. Its guard and assumptions determine whether a packet is applicable.

7.4 Chart

```
PiChart {
  id: ChartID
  context_id: ContextID
  prior_strength_p0: float
  prior_weight_k: float
  prior_provenance: str
```

```
  factorization_policy_id: PolicyID
  projection_policy_id: PolicyID
  kernel_projection_policy_id: PolicyID
  rule_profile_id: PolicyID
  selected_packet_ids: sorted[EvidencePacketID]
  evidence_snapshot_id: SnapshotID
  adequacy_certificate_id: str
  chart_fingerprint: str
  schema_version: int
}
```

7.5 Projection record

```
ProjectionRecord {
  id: ProjectionID
  statement: CanonicalTerm
  chart_id: ChartID
  packet_ids: sorted[EvidencePacketID]
  positive_count: float
  negative_count: float
  evidence_mass: float
  conflict_balance: float
  signed_tendency: float
  decision_stv: STV
  kernel_stv: STV
  beta_alpha: float
  beta_beta: float
  stamp_basis_ids: sorted[str]
  projection_policy_id: PolicyID
  kernel_projection_policy_id: PolicyID
  diagnostics: map[str, scalar]
}
```

7.6 Derived assessment

```
DerivedAssessment {
  id: str
  statement: CanonicalTerm
  chart_id: ChartID
  context_id: ContextID
  local_stv: STV
  decision_view: optional[DecisionView]
  primitive_token_ids: sorted[EvidenceTokenID]
  patham9_stamp: sorted[int]
  proof_path_id: ProofPathID
  proof_class_id: ProofClassID
  rule_identity_quality: EXACT | CANDIDATE_SET | UNKNOWN
  kernel_version: str
  rule_profile_id: PolicyID
  status: EPHEMERAL | REVIEWED | PROMOTED | REJECTED
}
```

A derived assessment is not an empirical packet.

8 Evidence algebra and overlap policy

8.1 Exact packet algebra

Let $\mathcal{U}$ be the set of evidence-basis identifiers. An exact evidence capsule is

$$e = (P^+, P^-, w^+, w^-, m), \tag{8.1}$$

where $P^+, P^- \subseteq \mathcal{U}$ are provenance sets, w^+, w^- map basis units to nonnegative weights, and m is metadata. Merge is provenance union with one contribution per basis unit:

$$e_1 \sqcup e_2 = \text{UnionByBasis}(e_1, e_2). \tag{8.2}$$

The valuation into count space is

$$\nu(e) = \left(\sum_{u \in P^+} w^+(u), \sum_{u \in P^-} w^-(u) \right). \tag{8.3}$$

Thus

$$\nu(e_1 \sqcup e_2) = \nu(e_1) + \nu(e_2) - \nu(e_1 \cap e_2), \tag{8.4}$$

with channel-specific overlap.

8.2 Evidence basis construction

The initial implementation SHOULD define one `EvidenceBasisUnit` per primitive token or per predeclared inseparable batch. A basis builder MUST produce:

```
EvidenceBasis {
  basis_id: str
  member_token_ids: sorted[EvidenceTokenID]
  causal_group_ids: sorted[str]
  independence_status: PROVEN_DISJOINT | ASSUMED | COUPLED | UNKNOWN
  justification_cid: str
}
```

Only `PROVEN_DISJOINT` basis units may be added without discount.

8.3 Hard and soft overlap decisions

Decision	Kernel behavior
Disjoint	Assign disjoint stamp aliases and permit the pair.
Exact overlap	Share one or more stamp aliases; stock patham9 rejects the binary application.
Partial overlap, no residualization	Reject the pair and record `OVERLAP_UNRESOLVED`.
Partial overlap, residualizable	Construct explicit disjoint residual evidence outside the kernel, reproject, and retry.

Approximate only	overlap	Use as a soft controller penalty; MUST NOT override the hard gate.
Unknown		Fail closed for evidence-combining rules; MAY allow non-evidential structural transforms under a distinct rule policy.

8.4 Episode-local stamp mapping

The compiler creates a deterministic bijection

$$a_E : \mathcal{U}_E \longrightarrow \{0, 1, \ldots, n_E - 1\} \tag{8.5}$$

for the episode basis set $\mathcal{U}_E$. Sorting is lexicographic by stable basis ID before integer assignment. The manifest stores both directions.

```
StampMapEntry {
  episode_id: EpisodeID
  stamp_int: int
  basis_id: str
  member_token_digest: str
}
```

The mapping MUST be collision-free by construction, not by truncated hashing.

8.5 Large provenance sets

If stamps become too large, the system MAY coarsen them only by creating explicit evidence-basis units whose semantics are recorded. Bloom filters or cardinality sketches MAY assist search and caching, but MUST NOT replace exact equality for the stock `StampDisjoint` gate.

9 Context atlas and chart lifecycle

9.1 Context candidate generation

The `ContextPlanner` takes a query, task state, candidate evidence, and budget, then produces bounded candidate guards. The initial implementation supports:

1. exact metadata filters, preserving current behavior;
2. bounded conjunction search over query and evidence features;
3. specialization by exception features;
4. query-specific covering contexts for cross-context comparison.

Candidate generation SHOULD use beam search with configured maximum depth, width, and feature count. Every candidate receives an inspectable certificate containing included features, excluded alternatives, and ablation results.

9.2 Chart utility

A reproducible scalarization MAY begin with

$$\begin{aligned} U(C, j) = \alpha s_{j,C} c_{j,C} + \beta A(C) + \gamma \Delta\,\mathrm{Conflict}(C) \\ - \delta\,\mathrm{Cost}(C, j) - \eta\,\mathrm{OverlapRisk}(C, j) - \zeta\,\mathrm{TranslationResidue}(C), \end{aligned} \tag{9.1}$$

where coefficients are versioned configuration. The underlying semantic label remains a product object; scalarization is only scheduling policy.

9.3 Chart lifecycle

1. **Propose**: construct a context and assumption package.
2. **Freeze evidence**: create an immutable packet snapshot.
3. **Validate**: check guard applicability, packet status, provenance, and chart assumptions.
4. **Project**: compute decision and kernel views.
5. **Compile**: generate one isolated patham9 program and episode manifest.
6. **Run**: execute through the kernel control port.
7. **Decode**: create proof and assessment records.
8. **Appraise**: compute evidence, cost, coherence, and splice diagnostics.
9. **Close**: persist the immutable episode result; discard or archive the ephemeral kernel space.
10. **Export**: remain ephemeral or enter a typed promotion/revision flow.

9.4 Covering contexts

Cross-context comparison requires a temporary object

```
CoveringContext {
  id: ContextID
  query_id: str
  source_context_ids: sorted[ContextID]
  translation_ids: sorted[str]
  common_language_fragment_id: str
  combined_assumption_package_id: str
  untranslated_residue_ids: sorted[str]
  compatibility_grade: float
  created_for_episode: EpisodeID
}
```

The covering context is local to a task. It MUST NOT silently replace the source contexts.

10 Projection policies and chart round trips

10.1 Canonical local chart projection

For empirical counts (n^+, n^-) and chart parameters $p_0 \in [0,1]$, $k > 0$,

$$N = n^+ + n^-, \qquad s = \frac{n^+ + kp_0}{N+k}, \qquad c = \frac{N}{N+k}, \tag{10.1}$$

and

$$\alpha = kp_0 + n^+, \qquad \beta = k(1-p_0) + n^-. \tag{10.2}$$

This is the `pipl-local-chart-v1` decision projection.

10.2 Conflict diagnostics

The projection record also computes

$$b = \frac{2\min(n^+, n^-)}{\max(N, \epsilon)}, \qquad t = \frac{n^+ - n^-}{\max(N, \epsilon)}. \tag{10.3}$$

N measures evidence mass, b balanced conflict, and t signed tendency. These MUST remain separate from confidence.

10.3 Kernel projection profiles

Policy	Intended use
`adapter-weighted-v1`	Reproduce the current wrapper baseline for differential tests.
`prior-aware-no-revision-v1`	Pass canonical prior-aware strength and confidence, preaggregate all same-term evidence, and filter stock revision outputs.
`empirical-frequency-v1`	Pass $s = n^+/N$ and $c = N/(N+k)$ so scalar formulas see an empirical frequency while the prior remains external.
`native-ec-v1`	Future profile used only if a kernel natively accepts evidence-count objects.

Decision and kernel projections MUST be recorded separately. Experiments MUST identify the profile used.

10.4 Beta chart round trip

A beta chart imports

$$(n^+, n^-) \mapsto \mathrm{Beta}(kp_0 + n^+, k(1-p_0) + n^-). \tag{10.4}$$

If the chart later exports a beta posterior known to have the same prior, the empirical counts are recovered by subtraction. This round trip is exact. If the output is an arbitrary derived STV or the prior has changed, export is not defined without a rule-specific policy.

Invariant 10.1 (No prior cycling)**.** *Creating, closing, and recreating an unchanged chart MUST leave the persistent evidence packet set and its valuation unchanged.*

11 Patham9 adapter

11.1 Adapter responsibilities

The `Patham9Adapter` MUST:

1. pin and report the patham9 version;
2. report kernel capabilities;
3. compile admitted assessments into exact `Sentence` atoms;
4. assign and persist the stamp map;
5. generate or load the requested rule profile;
6. run in an isolated process or space;
7. enforce time, memory, step, task-queue, and belief-queue limits;
8. capture stdout, stderr, return code, traces, and generated program;
9. validate output structure and numeric finiteness;
10. decode kernel outputs without claiming more rule identity than the trace supports;
11. never mutate the persistent evidence store.

11.2 Compiled sentence

```
(Sentence
  ((Inheritance (Concept Penguin) (Concept Bird))
   (stv 0.99 0.95))
  (3 8 21))
```

Every compiled sentence has a sidecar:

```
KernelSentenceMeta {
  episode_id: EpisodeID
  sentence_digest: str
  canonical_term: str
  projection_id: ProjectionID
  context_id: ContextID
  chart_id: ChartID
  stamp_ints: sorted[int]
  evidence_basis_ids: sorted[str]
}
```

11.3 Rule profiles

A `RuleProfile` declares allowed rule schemas and output handling. The initial profiles SHOULD include:

- `stock-all-v1`;

- `stock-minus-revision-v1`;
- `deduction-only-v1`;
- `safe-structural-v1`;
- `control-query-v1` for small meta-level controller queries.

With an unmodified stock kernel, a profile may be enforced by compiling a generated rule bundle or by filtering outputs. The former is preferred when exact exclusion matters.

11.4 Rule identity decoding

The current trace reports selected and derived sentences but not a stable rule identifier. The decoder therefore supports three grades:

1. `EXACT`: a tagged overlay or native event names the rule and binding;
2. `CANDIDATE_SET`: replay against the known rule library yields one or more compatible schemas;
3. `UNKNOWN`: the result cannot be attributed safely.

Promotion policies MAY require `EXACT`.

11.5 Isolation and determinism

An episode process MUST receive only:

- pinned kernel artifacts;
- generated chart-local sentences and rule definitions;
- explicit budgets;
- a deterministic seed if stochastic functionality is involved;
- no persistent write credentials.

The complete generated MeTTa program is stored in the episode manifest.

12 Kernel control port and forthcoming geodesic control

12.1 Control modes

The runtime supports three interchangeable modes.

Mode	Kernel change	Behavior
Legacy microstep	None	The outer controller chooses a task or small frontier batch and calls stock `PLN.Derive` with one or a few steps.

Overlay policy deriver	Separate overlay, vendor unchanged	A short MeTTa deriver exposes rank, compatibility, rule-allow, and event callbacks while reusing stock formulas and rules.
Native geodesic	Forthcoming patham9 capability	The kernel owns within-chart forward/backward frontiers and local f, g estimates, emitting standardized events through the port.

12.2 Capability record

```
KernelCapabilities {
  kernel_name: str
  kernel_version: str
  query: bool
  derive: bool
  pause_resume: bool
  checkpoint_restore: bool
  candidate_enumeration: bool
  custom_rank_callback: bool
  evidence_compatibility_callback: bool
  rule_allow_callback: bool
  derivation_event_stream: bool
  exact_rule_ids: bool
  forward_backward_frontiers: bool
  native_geodesic_score: bool
  native_cost_vector: bool
  stamp_passthrough: bool
}
```

Capabilities are probed, not inferred from a version string alone.

12.3 Python protocol

```
from typing import Iterable, Protocol

class KernelControlPort(Protocol):
    def capabilities(self) -> KernelCapabilities: ...

    def begin_episode(
        self,
        spec: KernelEpisodeSpec,
        observer: "KernelEventObserver",
    ) -> EpisodeHandle: ...

    def seed_forward(
        self,
        handle: EpisodeHandle,
        sentences: tuple[CompiledSentence, ...],
    ) -> None: ...

    def seed_backward(
        self,
        handle: EpisodeHandle,
```

```
        goals: tuple[KernelGoal, ...],
    ) -> None: ...

    def expand(
        self,
        handle: EpisodeHandle,
        request: ExpansionRequest,
    ) -> tuple[KernelTransition, ...]: ...

    def checkpoint(self, handle: EpisodeHandle) -> KernelCheckpoint: ...
    def restore(self, checkpoint: KernelCheckpoint) -> EpisodeHandle: ...
    def finish(self, handle: EpisodeHandle) -> KernelEpisodeResult: ...
    def abort(self, handle: EpisodeHandle, reason: str) -> None: ...
```

A legacy adapter may implement `seed_backward` and `checkpoint` as unsupported capabilities and emulate `expand` by a one-step process invocation.

12.4 Outer and inner controller contract

The outer atlas controller supplies an `InnerControlEnvelope`:

```
InnerControlEnvelope {
  chart_id: ChartID
  allowed_rule_profile_id: PolicyID
  forward_seed_ids: sorted[str]
  goal_terms: sorted[CanonicalTerm]
  max_steps: int
  max_expansions: int
  max_wall_ms: int
  max_memory_bytes: int
  local_cost_policy_id: PolicyID
  required_event_detail: SUMMARY | TRANSITIONS | FULL
  hard_evidence_gate: bool
  deterministic_mode: bool
  random_seed: int
}
```

The inner controller MAY choose local expansion order and local splices. It MUST NOT:

- choose or modify the chart;
- import packets outside the frozen snapshot;
- mint evidence;
- perform cross-context translation;
- promote results;
- mutate persistent memory;
- hide its selected path or cost summary from the outer runtime.

12.5 Hierarchical geodesic score

For a within-chart transition a and an atlas state z, the runtime uses

$$\begin{aligned}\mathrm{Score}(a \mid z) = {} & \Delta(\log f_{\mathrm{kernel}} + \log g_{\mathrm{kernel}}) + \Delta(\log f_{\mathrm{atlas}} + \log g_{\mathrm{atlas}}) \\ & - \lambda C_{\mathrm{compute}} - \eta C_{\mathrm{reuse}} - \zeta C_{\mathrm{curvature}} - \chi C_{\mathrm{translation}} - \omega C_{\mathrm{irreversibility}} \\ & + \beta \Delta I_{\mathrm{unique}} + \rho\,\mathrm{Fit} + \sigma\,\mathrm{Preservation} + \upsilon\,\mathrm{Weakness}\,.\end{aligned} \tag{12.1}$$

When the native kernel supplies local f, g and cost values, the outer controller consumes them as features. It does not reinterpret them as empirical evidence.

12.6 Macro-edge fallback

If a native backend exposes only completed paths, the outer runtime treats each returned path as a macro-edge. It can still perform provenance de-duplication, chart validation, endpoint appraisal, and cross-path holonomy tests. Fine-grained curvature penalties are unavailable and MUST be marked as such.

13 Outer atlas controller

13.1 Controller state

A controller state is

$$z = (C, \Gamma, x, [p]_{\mathsf{Ev}}, e, q, B, h), \tag{13.1}$$

where C is context, Γ chart, x proof endpoint, $[p]_{\mathsf{Ev}}$ evidence-aware proof component, e evidence capsule, q reversible residual/debt, B resource budget, and h audit history pointer.

13.2 Action types

The action alphabet includes:

- `KernelExpand`;
- `OpenChart`;
- `CloseChart`;
- `TranslateContext`;
- `ConstructCoveringContext`;
- `ReviseEvidence`;
- `ReviseContext`;
- `SpliceProofs`;
- `ProbeCurvature`;
- `AttemptDescent`;
- `Terminate`;
- `SubmitForPromotion`.

13.3 Budget vector

Budgets are multidimensional:

```
Budget {
  wall_ms: int
  cpu_ms: int
  memory_bytes: int
  kernel_steps: int
  chart_count: int
  translation_count: int
  revision_count: int
  curvature_probes: int
  proof_nodes: int
  exact_provenance_bytes: int
  privacy_budget: float | null
}
```

The controller MUST stop when any hard component is exhausted.

13.4 Forward and backward frontiers

The outer controller maintains:

- a forward frontier rooted in admitted premise assessments;
- a backward frontier rooted in query goals and context requirements;
- a chart frontier containing candidate contexts and translations;
- a revision frontier containing candidate corrections or refinements.

A splice is accepted only if endpoint terms unify, contexts are identical or covered, assumptions are compatible, evidence transport is coherent within tolerance, and provenance fusion is idempotent.

13.5 Initial f and g surrogates

The initial implementation MAY use:

$$\log f(z) = \theta_1 \log c_{\text{local}} + \theta_2 A(C) - \theta_3 \,\text{ForwardCost}(z), \tag{13.2}$$

$$\log g(z) = \phi_1 \,\text{GoalUnification}(z) + \phi_2 \,\text{BackwardReach}(z) - \phi_3 \,\text{RemainingCost}(z). \tag{13.3}$$

Later implementations may use Monte Carlo meet probabilities, factor-graph messages, learned value functions, or native patham9 bridge factors [9].

13.6 Admission plan

Before any kernel execution, the controller emits an immutable plan:

```
AdmissionPlan {
  id: str
  query_id: str
  candidate_chart_ids: sorted[ChartID]
  admitted_chart_ids: sorted[ChartID]
```

```
  rejected_candidates: list[Rejection]
  candidate_branch_ids: sorted[str]
  admitted_branch_ids: sorted[str]
  policy_id: PolicyID
  score_components: map[str, map[str, float]]
  budget: Budget
  random_seed: int
}
```

14 End-to-end query workflow

14.1 Query request

```
PiPLNQueryRequest {
  query_id: str
  target: CanonicalTerm
  task_context_hint: optional[ContextHint]
  desired_output: ASSESSMENT | PROOF | ALTERNATIVES | DECISION
  context_policy_id: PolicyID
  projection_policy_id: PolicyID
  controller_policy_id: PolicyID
  promotion_policy_id: PolicyID | null
  budget: Budget
  deterministic_mode: bool
  random_seed: int
}
```

14.2 Workflow

1. Validate the request and agent boundary.
2. Retrieve immutable packet candidates by statement pattern, context features, time, and task.
3. Generate and score candidate contexts.
4. Freeze an evidence snapshot for each admitted chart.
5. Aggregate packets using exact overlap policy.
6. Compute conflict diagnostics and decision projections.
7. Select a kernel projection and rule profile.
8. Compile one isolated episode per chart.
9. Delegate within-chart search through `KernelControlPort`.
10. Decode transitions into derived assessments and raw proof paths.
11. Map proof paths into evidence-aware proof classes.
12. Continue outer forward/backward search, including chart transitions if needed.

13. Perform splice, curvature, or descent checks required by policy.
14. Produce one of the typed results below.
15. Persist the audit artifact. Do not promote automatically.

14.3 Typed query results

```
PiPLNQueryResult =
    ResolvedAssessment
  | ContextualAlternatives
  | NonGluableCharts
  | InsufficientEvidence
  | BudgetExhausted
  | UnsafeOrUnreplayable
  | KernelFailure
```

Every result includes the chart/context identifiers, evidence packet IDs, proof IDs, policy versions, costs, conflict diagnostics, and kernel version.

15 Proof DAG, higher proof identity, and de-duplication

15.1 Rule application record

```
RuleApplication {
  id: KernelTransitionID
  episode_id: EpisodeID
  rule_ids: sorted[str]
  rule_identity_quality: EXACT | CANDIDATE_SET | UNKNOWN
  binding_digest: str
  premise_assessment_ids: sorted[str]
  conclusion_assessment_id: str
  input_stamp_sets: list[sorted[int]]
  output_stamp: sorted[int]
  chart_id: ChartID
  context_id: ContextID
  step_cost: CostVector
  kernel_event_cid: str
}
```

15.2 Raw path versus proof class

A raw proof path is an ordered list of rule applications. Its proof class key SHOULD include

$$K(p) = H(\,\mathrm{NormalForm}(\mathrm{conclusion}(p)), \mathrm{ChartOrCover}(p), \\ \mathrm{SortedPrimitiveBasis}(p), \mathrm{AssumptionFingerprint}(p), \mathrm{RuleSemanticsFingerprint}(p)). \tag{15.1}$$

Two paths with the same key MAY still be stored separately for explanation and control, but their evidence joins idempotently.

15.3 Rewrite cell

```
RewriteCell {
  id: RewriteCellID
  left_path_id: ProofPathID
  right_path_id: ProofPathID
  source_endpoint_digest: str
  target_endpoint_digest: str
  context_or_cover_id: ContextID
  semantic_identity_witness_id: str | null
  evidence_distance: float
  provenance_symmetric_difference: float
  assumption_difference: float
  coherence_grade: float
  status: EXACT | APPROXIMATE | REJECTED
}
```

A zero-defect cell witnesses that the two paths transport the same evidence. Approximate cells may be used for ranking but MUST NOT force exact idempotent equivalence unless the policy explicitly accepts the approximation.

15.4 Control mass and evidence mass

Let $\rho_{\mathrm{ctl}}(p)$ be search mass on raw paths and let $\pi(p)$ map to an evidence-aware proof class. Then

$$(\pi_*\rho_{\mathrm{ctl}})(C) = \sum_{p:\pi(p)=C} \rho_{\mathrm{ctl}}(p) \tag{15.2}$$

may be large even when the class contributes one evidence capsule. The runtime MUST report these quantities separately.

16 Reversible execution and audit

16.1 Operational meaning of reversibility

Logical implication is not assumed invertible. Reversibility means that the executed transition can be reconstructed and undone at the state-management level because the system retains what the visible step discards.

The implementation distinguishes:

1. **Replay reversibility**: regenerate the same output from immutable input, version, seed, and policy.
2. **State rollback**: restore the prior active pointer without deleting history.
3. **Algebraic inverse transport**: apply an explicit inverse map on an extended state; optional and rule-specific.

The first two are REQUIRED. The third is advanced.

16.2 Episode manifest

```
EpisodeManifest {
  episode_id: EpisodeID
  parent_episode_ids: sorted[EpisodeID]
  chart_id: ChartID
  context_id: ContextID
  evidence_snapshot_id: SnapshotID
  compiled_program_cid: str
  stamp_map_cid: str
  kernel_name: str
  kernel_version: str
  kernel_capabilities_cid: str
  rule_profile_id: PolicyID
  projection_policy_ids: sorted[PolicyID]
  controller_envelope_cid: str
  seed: int
  budget: Budget
  started_at: timestamp
  finished_at: timestamp
  return_code: int
  stdout_cid: str
  stderr_cid: str
  result_cid: str
}
```

16.3 Reversible complement

A transition complement MAY contain:

```
ReversibleComplement {
  prior_active_pointer: str
  complete_input_assessment_ids: sorted[str]
  overwritten_local_values: map[str, value]
  chart_prior_and_assumptions_cid: str
  overlap_witness_cid: str
  bregman_residual_cid: str | null
  dykstra_increment_cid: str | null
  translation_residue_cid: str | null
  rng_state_cid: str | null
}
```

16.4 Bregman residual

If a projection-like operation has Legendre potential Φ, visible output $y = P^{\Phi}_{\mathcal{M}}(x)$, and dual residual

$$q = \nabla\Phi(x) - \nabla\Phi(y), \tag{16.1}$$

then

$$x = \nabla\Phi^*(\nabla\Phi(y) + q). \tag{16.2}$$

The runtime MAY store q as a geometrically meaningful reversible complement. For legacy scalar formulas whose projection geometry is not established, complete input records are the authoritative complement.

16.5 Append-only active state

Persistent mutation uses versioned pointers:

```
ActivePointer {
  logical_key: str
  active_record_cid: str
  previous_pointer_cid: str | null
  changed_by_revision_id: RevisionID | null
  changed_by_promotion_id: PromotionID | null
}
```

Undo creates a new pointer to an earlier record; it does not erase events.

17 Revision as a first-class action

17.1 Revision kinds

```
RevisionKind =
    ADD_INDEPENDENT_EVIDENCE
  | DISCOUNT_PACKET
  | QUARANTINE_PACKET
  | RETRACT_PACKET
  | RELABEL_PACKET
  | TRANSLATE_CONTEXT
  | SPLIT_CONTEXT
  | MERGE_CONTEXTS
  | CHANGE_CHART_PRIOR
  | CHANGE_VALUATION
  | CHANGE_ONTOLOGY
```

Discovering that old support was duplicated or invalid normally changes or quarantines that support. It does not automatically create negative evidence for the object-level statement.

17.2 Revision scoring

For current local state H, new evidence E, and candidate revised state K, a product score contains

$$(\mathrm{Fit}(K;E), \mathrm{Pres}(K;H), \mathrm{Weak}(K), -\mathrm{Cost}(K)). \tag{17.1}$$

A scalar scheduler MAY use lexicographic ordering or a versioned scalarization. The revision record MUST retain the full component vector.

17.3 Revision record

```
RevisionRecord {
  id: RevisionID
  kind: RevisionKind
  old_record_cids: sorted[str]
  evidence_packet_ids: sorted[EvidencePacketID]
  candidate_record_cids: sorted[str]
  selected_record_cid: str
```

```
  source_context_ids: sorted[ContextID]
  target_context_id: ContextID
  fit_grade: float
  preservation_grade: float
  weakness_grade: float
  cost_vector: CostVector
  reversible_complement_cid: str
  policy_id: PolicyID
  reviewer_id: str | null
}
```

18 Curvature, commutator, holonomy, and descent diagnostics

18.1 Sparse confusion graph

A pair of rule or context actions is a curvature-probe candidate when at least one condition holds:

- their primitive provenance overlaps;
- they write the same conclusion or local value;
- they use the same nonlinear truth-value state;
- they share assumptions or a context translation;
- they are known to interact through a rule-specific declaration;
- a learned predictor assigns high interaction probability.

The graph SHOULD be sparse. Disjoint pairs need no probe.

18.2 Discrete commutator probe

Given a frozen reversible microstate z and actions r, s, run both orders in isolated sandboxes:

$$z_{rs} = U_s U_r z, \qquad z_{sr} = U_r U_s z. \tag{18.1}$$

Define

$$\delta_{rs}(z) = d_{\mathsf{Ev}}(z_{rs}, z_{sr}). \tag{18.2}$$

If an order is not applicable, the result is `NOT_COMPARABLE`, not zero.

18.3 Evidence-state metric

A configurable metric MAY combine

$$\begin{aligned} d^2_{\mathsf{Ev}}(z, z') = \lambda_{\mathrm{tv}} d^2_{\mathrm{chart}}(z, z') + \lambda_{\mathrm{prov}} \mu(P_z \triangle P_{z'}) \\ + \lambda_{\mathrm{ctx}} d^2_{\mathrm{ctx}}(C_z, C_{z'}) + \lambda_{\mathrm{assump}} \mathbf{1}[A_z \neq A_{z'}] + \lambda_q \left\| q_z - q_{z'} \right\|^2 . \end{aligned} \tag{18.3}$$

Inside a common beta chart, d_{chart} SHOULD be a distributional or Fisher-compatible distance. Outside a common chart, the metric MUST include translation residue and MUST NOT compare scalar STVs as if they were commensurate.

18.4 Curvature probe record

```
CurvatureProbe {
  id: str
  base_state_cid: str
  first_action_id: str
  second_action_id: str
  result_rs_cid: str | null
  result_sr_cid: str | null
  evidence_distance: float | null
  metric_policy_id: PolicyID
  comparability: COMPARABLE | NOT_COMPARABLE | FAILED
  cache_key: str
}
```

18.5 Proof-path bigon and relative holonomy

For two complete paths p, q with common typed endpoints, define the observed bigon defect

$$\mathcal{H}(p, q; e) = \frac{1}{2} d_{\mathsf{Ev}}(T_p e, T_q e)^2. \tag{18.4}$$

If reversible transports are available, define

$$H_{p,q} = \widetilde{T}_q^{-1} \widetilde{T}_p, \tag{18.5}$$

and record the displacement of the evidence state under $H_{p,q}$. This is an executable holonomy diagnostic, not an additional truth value.

18.6 Context and mixed curvature

For translation $\tau : C \to D$ and rule r, compare infer-then-translate with translate-then-infer:

$$\delta_{\tau,r}(e) = d\left(T_\tau T_r(e), T_{\tau_* r} T_\tau(e)\right). \tag{18.6}$$

For context translations $C \xrightarrow{\tau} D \xrightarrow{\sigma} E$, compare

$$\delta_{\sigma,\tau}(e) = d\left(T_{\sigma\circ\tau}(e), T_\sigma T_\tau(e)\right). \tag{18.7}$$

These are stored separately from within-chart proof curvature.

18.7 Descent obstruction

Given local charts ν_i and restriction operators r_i, a chart-gluing service MAY compute

$$\mathcal{O}_{\mathrm{desc}}(\nu_\bullet) = \inf_{\nu} \sum_i d_i(r_i \nu, \nu_i). \tag{18.8}$$

A positive obstruction means no exact global chart exists in the chosen model class. This is not the same as path curvature and MUST be reported separately.

19 Promotion and export

19.1 Rule-specific export policies

Every rule schema has an explicit policy:

```
EvidenceExportPolicy {
  rule_id: str
  output_kind: DERIVED_ONLY | EMPIRICAL_INCREMENT | HYPOTHESIS
  may_mint_token: bool
  required_rule_identity_quality: EXACT | CANDIDATE_SET
  prior_unwind_required: bool
  overlap_policy_id: PolicyID
  reviewer_required: bool
}
```

Typical initial settings are:

Rule or source	Export behavior
Direct external observation	May mint a primitive token under observation validation.
Same-context packet revision	Union or correct existing packet provenance.
Deduction / modus ponens	Derived assessment only.
Induction / abduction	Hypothesis only.
Context translation	Derived or translated packet only with explicit witness and reliability.
Unknown rule identity	Never promote.

19.2 Promotion gate

A promotion request MUST contain:

- exact target statement and context;
- complete proof and replay manifest;
- rule and formula versions;
- chart, assumptions, and prior data;
- exact primitive provenance;
- overlap analysis;
- export policy;
- idempotence key;
- reviewer or policy authorization when required.

Promotion output is one of:

- `REJECT`;
- `ARTIFACT_ONLY`;
- `PROMOTE_DERIVED`;
- `PROMOTE_EVIDENCE_PACKET`.

20 Service interfaces

20.1 Evidence store protocol

```
class ContextualEvidenceStore(Protocol):
    def create_snapshot(
        self,
        query: EvidenceQuery,
        context_id: ContextID,
    ) -> EvidenceSnapshot: ...

    def aggregate(
        self,
        snapshot: EvidenceSnapshot,
        statement: CanonicalTerm,
        policy: OverlapPolicy,
    ) -> EvidenceAggregate: ...

    def apply_revision(
        self,
        request: RevisionRequest,
    ) -> RevisionRecord: ...

    def get_packet(self, packet_id: EvidencePacketID) -> EvidencePacket: ...
```

20.2 Context and chart protocols

```
class ContextPlanner(Protocol):
    def propose(
        self,
        query: PiPLNQueryRequest,
        evidence_index: EvidenceFeatureIndex,
        budget: Budget,
    ) -> tuple[ContextCandidate, ...]: ...

class ChartBuilder(Protocol):
    def build(
        self,
        candidate: ContextCandidate,
        snapshot: EvidenceSnapshot,
        policy: ChartPolicy,
    ) -> PiChart: ...

class ChartTranslationService(Protocol):
    def construct_cover(
        self,
        query_id: str,
        contexts: tuple[ContextID, ...],
        budget: Budget,
    ) -> CoveringContextResult: ...
```

20.3 Projection protocol

```
class ProjectionPolicy(Protocol):
    @property
    def policy_id(self) -> PolicyID: ...

    def project_decision(
        self,
        aggregate: EvidenceAggregate,
        chart: PiChart,
    ) -> DecisionProjection: ...

    def project_kernel(
        self,
        aggregate: EvidenceAggregate,
        chart: PiChart,
        rule_profile: RuleProfile,
    ) -> KernelProjection: ...
```

20.4 Controller protocol

```
class AtlasController(Protocol):
    def plan(self, request: PiPLNQueryRequest) -> AdmissionPlan: ...

    def run(
        self,
        request: PiPLNQueryRequest,
        plan: AdmissionPlan,
        kernel: KernelControlPort,
    ) -> PiPLNQueryResult: ...
```

20.5 Proof store protocol

```
class ProofStore(Protocol):
    def add_transition(self, transition: RuleApplication) -> str: ...
    def add_path(self, path: ProofPath) -> ProofPathID: ...
    def classify(self, path_id: ProofPathID) -> ProofClassID: ...
    def add_rewrite_cell(self, cell: RewriteCell) -> RewriteCellID: ...
    def provenance_join(self, class_id: ProofClassID) -> EvidenceCapsule: ...
```

21 MeTTa annotation layer

21.1 Evidence and chart annotations

```
(PiEvidenceToken token-17
  (Namespace sensor-A)
  (Source source-5)
  (ObservedIn context-zoological)
  (ObservedAt time-2026-07-12T10:00:00Z)
```

```
  (CausalGroup event-44)
  (Status Active))

(PiEvidencePacket packet-44
  (Statement (Inheritance Penguin Bird))
  (Context context-zoological)
  (Counts 1.0 0.0)
  (Tokens (token-17))
  (Assumptions assumptions-zoo-v3)
  (Status Active))

(PiChart chart-91
  (Context context-penguin-flight)
  (Prior 0.50 2.0)
  (ProjectionPolicy pipl-local-chart-v1)
  (KernelProjectionPolicy empirical-frequency-v1)
  (RuleProfile stock-minus-revision-v1)
  (Snapshot snapshot-771))
```

21.2 Episode annotations

```
(PiEpisode episode-22
  (Chart chart-91)
  (Kernel patham9-main-snapshot)
  (Seed 381919)
  (MaxSteps 1)
  (TaskQueueSize 20)
  (BeliefQueueSize 200))

(PiStampMap episode-22 0 evidence-basis-17)
(PiStampMap episode-22 1 evidence-basis-31)
```

21.3 Proof and curvature annotations

```
(PiRuleApplication app-71
  (Episode episode-22)
  (RuleCandidates (modus-ponens))
  (Premises (assessment-12 assessment-18))
  (Conclusion assessment-209)
  (InputStamps ((0) (1)))
  (OutputStamp (0 1)))

(PiRewriteCell cell-44
  (LeftPath proof-r-then-s)
  (RightPath proof-s-then-r)
  (Context chart-91)
  (EvidenceDistance 0.003)
  (Status ApproximatelyCoherent))
```

22 Persistence, concurrency, and distribution

22.1 Recommended physical layout

The architecture SHOULD combine:

- a content-addressed immutable object store for packet bodies, manifests, traces, programs, and proofs;
- an indexed database for context, statement, source, time, and active-pointer queries;
- AtomSpace annotations pointing to stable object IDs;
- append-only event streams for revision and promotion;
- optional MORK-compatible Merkle-DAG replication for distributed evidence capsules [9].

22.2 Concurrency model

Episodes are isolated actors. They may read one frozen snapshot and write only new immutable objects. Persistent active-pointer changes use optimistic concurrency:

1. read pointer version;
2. construct immutable candidate update;
3. compare-and-swap the pointer;
4. on conflict, recompute against the new snapshot.

Idempotence keys make retries safe.

22.3 Caching

Safe caches include:

- evidence aggregate by snapshot, statement, and overlap policy;
- projection by aggregate digest, chart, and projection policy;
- compiled episode by chart fingerprint and sentence digest;
- kernel result by episode fingerprint;
- curvature probe by base-state fingerprint, ordered action pair, and metric policy;
- proof class by normalized endpoint, provenance digest, context/cover, assumptions, and semantics fingerprint.

Caches are content-addressed; invalidation is replacement by a new key.

22.4 Privacy

Exact provenance can be sensitive. A deployment MUST define:

- which token fields are plaintext, encrypted, or access-controlled;
- retention periods for reversible complements;
- whether privacy budgets limit cross-context joins;
- how deliberate irreversible erasure is authorized and audited;
- whether remote controllers receive raw provenance or only signed overlap results.

23 Security and agent-boundary controls

23.1 Read/write separation

Kernel workers have no credentials for persistent evidence mutation. Promotion and revision services are separate processes with narrower schemas and authorization.

23.2 Input hardening

The compiler MUST:

- canonicalize and validate terms;
- reject unexpected executable forms;
- whitelist rule profiles and imports;
- escape or isolate user-supplied symbols;
- bound generated program size;
- record the exact emitted program.

23.3 Output hardening

The runtime gate checks:

- return code and timeout status;
- output schema and finite numeric values;
- expected number of selected and derived records;
- consistency of stamps and sidecar mappings;
- absence of false/error markers under the configured test contract;
- absence of directive-like or task-execution payloads in appraisal artifacts;
- proof and provenance completeness required by the result type.

23.4 Controller circularity

A meta-level PLN controller MUST run in a separate chart and budget from the object-level query. Its evidence and proof trail remain separately auditable. It cannot promote its own branch-viability judgment as object-level evidence.

24 Observability and audit

24.1 Required event stream

The runtime emits structured events:

```
QUERY_ACCEPTED
CONTEXT_CANDIDATE
CONTEXT_SELECTED
SNAPSHOT_FROZEN
PROJECTION_CREATED
EPISODE_COMPILED
KERNEL_EPISODE_STARTED
KERNEL_TASK_SELECTED
KERNEL_TRANSITION_DERIVED
KERNEL_EPISODE_FINISHED
PROOF_PATH_CREATED
PROOF_CLASS_ASSIGNED
REWRITE_CELL_EVALUATED
CURVATURE_PROBE_COMPLETED
DESCENT_ATTEMPT_COMPLETED
REVISION_PROPOSED
PROMOTION_DECIDED
QUERY_FINISHED
```

Every event carries query, episode, chart, policy, and causal trace IDs where applicable.

24.2 Metrics

At minimum, report:

- packets retrieved and admitted;
- exact overlap rejections;
- evidence mass, conflict balance, and signed tendency;
- charts proposed, admitted, and rejected;
- kernel steps, queue sizes, and wall time;
- raw proof paths and evidence-aware proof classes;
- control mass per proof class;
- unique evidence mass per proof class;
- curvature probes and nonzero defects;

- splice attempts and mismatch reasons;
- chart gluing status and descent obstruction;
- replay success and manifest completeness;
- promotion requests and rejection reasons.

25 Testing and verification

25.1 Test layers

Layer	Required tests
Unit	Dataclass validation, projections, overlap, context guards, fingerprints, score components.
Property	Evidence join laws, prior round trip, deterministic stamp assignment, idempotence keys, revision rollback.
Differential	Compatibility mode versus stock patham9 examples and rule tests.
Metamorphic	Permuting disjoint evidence or commuting rule steps does not change the result.
Integration	Complete query through isolated kernel, trace decode, proof class, audit manifest.
Security	Malformed terms, nonfinite STVs, oversized programs, timeout, output injection, unauthorized promotion.
Replay	Re-execute stored manifests and compare result and trace digests.
Controller	Fixed-seed admission plans, budget exhaustion, native/legacy backend parity on simple cases.
Geometry	Zero defect for declared disjoint pairs, positive defect for constructed confusing pairs, non-gluable chart example.
Chaos	Worker crash, duplicate event delivery, retry, stale pointer, partial object replication.

25.2 Core algebraic properties

For evidence capsules a, b, c:

$$a \sqcup a = a, \tag{25.1}$$

$$a \sqcup b = b \sqcup a, \tag{25.2}$$

$$(a \sqcup b) \sqcup c = a \sqcup (b \sqcup c). \tag{25.3}$$

For chart import/export under a fixed beta prior:

$$O_C I_C(n^+, n^-) = (n^+, n^-). \tag{25.4}$$

For deterministic stamp construction:

$$\mathrm{StampMap}(S) = \mathrm{StampMap}(\mathrm{permute}(S)). \tag{25.5}$$

25.3 Acceptance scenarios

Scenario	Required result
$(1, 1)$ versus $(1000, 1000)$	Similar empirical strength, radically different mass/conflict annotations.
Prior cycling	No packet or count change after repeated chart construction.
Same source, two proof paths	Two control paths, one evidence contribution.
Disjoint sources	Counts may combine under the declared basis policy.
Partial overlap	Reject or residualize explicitly.
Same surface term, different contexts	No merge without covering context.
Non-gluable anti-correlation charts	Return non-gluability object.
Invalidated support	Quarantine/retract; do not create opposition automatically.
Derived result replay	Same output, stamp, policy, and trace digest.
Duplicate promotion	No active-state change.
Disjoint action pair	Commutator probe zero or below tolerance.
Confusing action pair	Nonzero defect recorded and available to scoring.
Legacy/native controller simple case	Equivalent accepted proof and evidence class under matched policy.
Compatibility mode	Stock patham9 tests and examples unchanged.

25.4 Patham9 regression command

The pinned vendor test harness runs the repository rule tests and examples. The extension test suite MUST run these before its own integration tests [6].

26 Performance and scalability

26.1 Expected cost centers

The dominant costs are likely to be:

- context candidate generation;
- exact provenance set operations;
- repeated isolated kernel startup in legacy microstep mode;
- proof-class normalization;
- curvature probes, which require two ordered sandbox executions;
- chart-gluing optimization.

26.2 Scaling strategy

1. Cache content-addressed projections and episode results.
2. Batch compatible microsteps within one chart when it does not reduce controller visibility.

3. Use exact provenance only on the active frontier and compact archival forms elsewhere.
4. Build a sparse confusion graph before curvature probing.
5. Perform descent attempts only when a query demands a cross-chart global answer.
6. Delegate within-chart search to native patham9 geodesic control when available.
7. Use worker pools keyed by chart fingerprint to amortize process startup without chart leakage.

26.3 Benchmark dimensions

Benchmarks MUST report semantic quality and cost jointly:

- query accuracy and calibration;
- false confidence from evidence reuse;
- conflict preservation;
- ontology-smear errors;
- proof nodes expanded;
- wall and CPU time;
- provenance bytes and proof-store growth;
- path-order sensitivity;
- controller overhead relative to stock patham9;
- reproducibility rate;
- native versus legacy geodesic backend efficiency.

27 Migration from the current partial effort

27.1 Function-to-component mapping

Current wrapper function or concept	Target component
`ec_projected_stv()`	Projection registry entry `adapter-weighted-v1`; retained for baseline comparison.
`context_selection_wrapper()`	Exact-filter backend inside `ContextPlanner`.
`probabilistic_inference_filter()`	Feature in `AtlasController` scoring, not a semantic truth rule.
`chained_inference_pipeline()`	`PiPLNQueryOrchestrator`.
`continuation_predicate_wrapper()`	Versioned `ContinuationPolicy`.
`controlled_backward_chainer()`	Legacy implementation of `KernelControlPort` plus outer agenda.
`pln_estimator_wrapper()`	`BranchValueEstimator`; beta/Thompson backend.
`ranked_inference_control_plan()`	`AdmissionPlanBuilder`.

`ranked_plan_admitted_handoff()`	`ChartedEpisodeCompiler`.
`controller_as_chainer()`	Separate `MetaControllerAdapter` with its own chart and audit trail.
Numeric stamps plus JSON sidecar	Deterministic episode `StampMap` plus content-addressed manifest.
Runtime proof gate	General `KernelResultValidator` and promotion precondition.

27.2 Compatibility preservation

The first migration release SHOULD wrap existing public functions rather than delete them. Each old function receives a deprecation note and delegates to the new component with a compatibility policy. Existing serialized artifacts remain readable through versioned parsers.

27.3 Data migration

Existing evidence sidecars are imported as packets with:

- explicit context IDs derived from current domain/cluster/rule fields;
- a conservative basis unit per current numeric provenance item;
- `independence_status=UNKNOWN` unless disjointness is established;
- projection policy `adapter-weighted-v1`;
- original artifact CIDs attached for audit.

No inferred STV is converted to empirical counts during migration.

28 Repository layout

```
project/
  vendor/
    patham9-pln/                # pinned, read-only by default

  pipln/
    metta/
      PiAnnotations.metta
      PiProjection.metta
      PiContext.metta
      PiIdentity.metta
      PiRevision.metta
      PiExplain.metta
      PiTaggedRules.metta       # optional exact rule trace
      PiDeriver.metta           # optional policy-hook overlay

    runtime/
      models.py
      ids.py
      evidence_store.py
      provenance.py
```

```
      context_planner.py
      chart_builder.py
      projection.py
      episode_compiler.py
      kernel_port.py
      patham9_legacy_backend.py
      patham9_overlay_backend.py
      patham9_native_geodesic_backend.py
      trace_decoder.py
      proof_store.py
      proof_classes.py
      atlas_controller.py
      value_estimator.py
      curvature.py
      descent.py
      revision.py
      promotion.py
      audit.py
      security.py

    schemas/
      evidence-token.schema.json
      evidence-packet.schema.json
      context.schema.json
      chart.schema.json
      episode.schema.json
      proof.schema.json
      revision.schema.json
      query-result.schema.json

    tests/
      unit/
      property/
      differential/
      integration/
      security/
      replay/
      benchmarks/

    docs/
      ADRs/
      CODING_AGENTS.md
      implementation-status.md

  tools/
    build_patham9.sh
    run_vendor_tests.sh
    replay_episode.py
    inspect_provenance.py
    probe_curvature.py
```

29 Implementation roadmap

29.1 Phase 0: baseline freeze

Deliverables

- pin vendor commit and runtime versions;
- capture stock build, rule-test, and example outputs;
- wrap current partial functions in a compatibility package;
- define content hashes for generated programs and artifacts.

Exit criteria: compatibility tests pass and every current smoke artifact is reproducible.

29.2 Phase 1: evidence, contexts, charts, and projections

Deliverables

- typed records and JSON schemas;
- immutable evidence store and snapshots;
- exact overlap and deterministic stamp basis;
- explicit `ChartPolicy`;
- canonical STV/beta projection and conflict diagnostics;
- compatibility projection retained.

Exit criteria: algebraic properties, prior cycling, conflict/ignorance, and stamp tests pass.

29.3 Phase 2: isolated patham9 episodes

Deliverables

- episode compiler;
- isolated legacy backend;
- structured trace decoder;
- kernel result validator;
- complete episode manifest and replay tool.

Exit criteria: stock examples pass through the adapter; exact replay succeeds.

29.4 Phase 3: external geodesic controller

Deliverables

- outer agenda and budget vector;
- forward/backward frontier objects;
- beta/Thompson and deterministic value estimators;
- immutable admission plans;
- one-step or small-microbatch kernel expansion;
- splice validation.

Exit criteria: declared search-cost or precision metric improves on at least one benchmark without changing compatibility-mode semantic answers.

29.5 Phase 4: proof geometry and reversibility

Deliverables

- proof DAG and proof classes;
- idempotent evidence fusion by proof class;
- optional tagged rule overlay;
- reversible complements and active-pointer rollback;
- duplicate-promotion gate.

Exit criteria: duplicate paths do not inflate evidence; every result is replayable and every revision reversible by pointer history.

29.6 Phase 5: atlas transitions and revision

Deliverables

- generated contexts and adequacy certificates;
- context maps, covering contexts, and untranslated residue;
- packet-level corrective revision;
- chart lifecycle and cross-chart query results.

Exit criteria: ontology-boundary and context-misselection scenarios return typed, inspectable results.

29.7 Phase 6: native patham9 geodesic integration

Deliverables

- capability probe;
- `Patham9NativeGeodesicBackend`;
- mapping of native events, f/g values, costs, checkpoints, and splices into the kernel port;
- parity tests against the legacy backend;
- controller-ownership tests preventing cross-layer mutation.

Exit criteria: the same outer query workflow can switch backends by configuration, with evidence and audit invariants unchanged.

29.8 Phase 7: curvature and descent

Deliverables

- sparse confusion graph;
- cached pairwise commutator probes;
- proof bigon and splice-holonomy records;
- context/mixed curvature probes;
- non-gluable chart benchmark and descent service.

Exit criteria: diagnostics distinguish evidence multiplicity, path holonomy, and chart non-gluability.

29.9 Phase 8: reviewed promotion

Deliverables

- rule-specific export policy registry;
- promotion review API;
- append-only persistent update events;
- rollback and duplicate-promotion tests.

Exit criteria: durable updates occur only through typed, audited, idempotent promotion.

30 Risks and open design questions

30.1 Semantic compression

Scalar STVs cannot carry all context, conflict, dependence, and provenance structure. The design contains this loss by restricting it to one charted episode and preserving the richer outer state. Some rule outputs may remain too lossy for safe export.

30.2 Evidence dependence

Exact token non-overlap does not prove statistical or causal independence. Causal-group annotations and conservative policies reduce the risk, but learning a correct causal provenance quotient remains open.

30.3 Context combinatorics

Generated guards can grow exponentially. Beam bounds, feature limits, caching, and minimality certificates are required. Context discovery quality may dominate inference quality.

30.4 Native geodesic API uncertainty

The forthcoming patham9 interface may expose different primitives than anticipated. Capability negotiation and macro-edge fallback prevent this from becoming an architectural dependency.

30.5 Rule identity

Current trace granularity may leave multiple possible rule schemas. Exact promotion may require the tagged overlay or a small upstream event seam.

30.6 Controller interaction

An outer and inner geodesic controller can fight if both independently rescore the same local decisions. The control envelope and ownership table are intended to prevent this. Experiments must test whether outer penalties can be pushed into the native inner cost model without semantic drift.

30.7 Reversible storage cost

Exact replay and rollback retain large programs, traces, and provenance. Content addressing, shared DAG nodes, selective compression, and retention policy are required. Deliberate erasure must be explicit because it weakens reversibility.

30.8 Curvature metric choice

A discrete order defect depends on the chosen evidence-state metric. Every metric is a versioned policy and results are not comparable across policies without calibration.

30.9 Chart gluing model class

A descent obstruction is relative to a candidate global model class. Adding hidden variables can remove an obstruction. The system should report the class and optimization tolerance, not a metaphysical claim of impossibility.

31 Summary of the implementation contract

The broad πPLN conception can be implemented on top of patham9 without forcing its rich semantics into the narrow STV field. The durable system is the atlas, evidence store, proof geometry, controller, and audit log. patham9 is a chart-local rule engine.

The minimal architecture is summarized by

$$\boxed{\begin{array}{c}\text{persistent packet evidence} \longrightarrow \text{context and chart selection} \\ \longrightarrow \text{temporary scalar projection} \longrightarrow \text{bounded patham9 inference} \\ \longrightarrow \text{proof-class de-duplication and coherence appraisal} \\ \longrightarrow \text{explicit revision or promotion.}\end{array}}$$

The geodesic extension is hierarchical:

$$\boxed{\text{outer atlas geodesic control} \supset \text{inner chart-local patham9 geodesic control.}} \tag{31.1}$$

The outer controller selects the temporary world in which inference is meaningful; the inner controller searches efficiently inside it. Evidence conservation, context typing, and promotion authority remain outside the kernel in every mode.

A Coding-agent operating guide

A.1 Mandatory first steps

Before editing code, a coding agent MUST:

1. read this specification section relevant to the assigned work package;
2. read all architectural decision records named in the task;
3. inspect the current implementation and tests rather than relying on the task summary alone;
4. identify the pinned patham9 version and verify whether the vendor tree is read-only for the task;
5. state the invariants the change could affect;
6. write or identify a failing test before implementing semantic behavior;
7. record uncertainties and stop when an ambiguity could affect evidence, context, promotion, or security semantics.

A.2 Absolute prohibitions

A coding agent MUST NOT:

1. modify `vendor/patham9-pln` unless the issue explicitly authorizes a vendor patch;
2. change the positional shape of patham9 `Sentence`;
3. encode controller utility by modifying semantic confidence;
4. add evidence counts without a provenance-disjointness or overlap policy result;
5. mint a primitive token from deduction, induction, abduction, or modus ponens;
6. convert an arbitrary derived STV into empirical counts;
7. merge different contexts by string equality of terms;

8. persist a chart prior as evidence;
9. make promotion automatic while the promotion policy is absent or incomplete;
10. use an approximate sketch to override a hard overlap rejection;
11. introduce nondeterministic tests without recording a seed;
12. claim a proposed or simulated native geodesic feature is implemented in public patham9;
13. delete history to implement undo;
14. relax a fail-closed check merely to make a test pass.

A.3 Task packet template

Every coding task SHOULD be issued with:

```
Title:
Goal:
In scope:
Out of scope:
Normative invariants:
Files allowed to change:
Vendor changes allowed: yes/no
Schemas affected:
Backward compatibility requirement:
Tests to add:
Acceptance command:
Artifacts to attach:
Stop conditions:
```

A.4 Implementation sequence

Agents SHOULD work in this order:

1. model and schema;
2. validation;
3. pure domain function;
4. property and unit tests;
5. storage adapter;
6. orchestration wiring;
7. integration test;
8. documentation and status update;
9. replay or migration test if persistent artifacts change.

A.5 Definition of done

A task is complete only when:

1. all new and existing relevant tests pass;
2. the vendor regression suite passes when patham9 behavior is touched;
3. new records carry schema versions and validation;
4. deterministic artifacts include seeds and fingerprints;
5. evidence/context/promotion invariants are tested directly;
6. public interfaces and error cases are documented;
7. no placeholder silently falls back to unsafe behavior;
8. implementation status is updated as implemented, partial, experimental, or proposed;
9. the pull request includes exact commands and output summaries used for validation.

A.6 Review checklist

Reviewers and review agents MUST ask:

- Can this change fabricate, duplicate, erase, or relabel evidence?
- Can it merge contexts or priors incorrectly?
- Can it make a non-replayable result appear promotion-ready?
- Does it conflate controller score, confidence, evidence mass, or conflict?
- Does it depend on an unreleased kernel API without a capability fallback?
- Is the cache key missing a chart, policy, snapshot, seed, or kernel version?
- Does retry preserve idempotence?
- Does failure leave an active pointer or partial persistent write?
- Are approximate and exact statuses clearly distinguished?
- Could untrusted content become executable MeTTa or an agent directive?

A.7 Coding-agent work decomposition

Large phases SHOULD be divided among agents with non-overlapping authority:

Agent role	Responsibility
Domain-model agent	Typed records, schemas, validation, hashes, migrations.
Evidence agent	Packet algebra, provenance basis, overlap, stamps, conflict diagnostics.

Kernel agent	Episode compiler, patham9 process adapter, trace parser, vendor regression.
Control agent	Admission plans, budgets, frontiers, value estimators, backend port.
Proof agent	Proof DAG, proof classes, rewrite cells, de-duplication.
Reversibility agent	Manifests, complements, replay, active-pointer rollback.
Atlas agent	Context generation, chart builder, translations, covering contexts.
Geometry agent	Confusion graph, commutator probes, holonomy and descent records.
Security agent	Sandbox, schemas, output gate, agent-boundary checks.
Evaluation agent	Differential, property, integration, benchmark, and chaos tests.

Shared files require an explicit integration owner.

A.8 Agent stop conditions

An agent MUST stop and request clarification when:

- a task requires deciding whether two sources are independent without policy data;
- a schema change would reinterpret existing evidence;
- a patham9 trace is insufficient to identify a rule required by promotion;
- a cross-context merge lacks a translation witness;
- the requested behavior implies generic STV-to-count inversion;
- a future native geodesic API is assumed but unavailable;
- a security check conflicts with a convenience path;
- tests reveal that wrapper behavior cannot preserve a required invariant.

B Kernel capability adapter examples

B.1 Legacy capability record

```
LEGACY_PATHAM9_CAPABILITIES = KernelCapabilities(
    kernel_name="patham9-pln",
    kernel_version="pinned-main-snapshot",
    query=True,
    derive=True,
    pause_resume=False,
    checkpoint_restore=False,
    candidate_enumeration=False,
    custom_rank_callback=False,
    evidence_compatibility_callback=False,
    rule_allow_callback=False,
    derivation_event_stream=False,
    exact_rule_ids=False,
    forward_backward_frontiers=False,
    native_geodesic_score=False,
```

```
    native_cost_vector=False,
    stamp_passthrough=True,
)
```

B.2 Hypothetical native capability record

This example is a target contract, not a claim about a released API.

```
NATIVE_GIC_CAPABILITIES = KernelCapabilities(
    kernel_name="patham9-pln",
    kernel_version="future-native-gic",
    query=True,
    derive=True,
    pause_resume=True,
    checkpoint_restore=True,
    candidate_enumeration=True,
    custom_rank_callback=True,
    evidence_compatibility_callback=True,
    rule_allow_callback=True,
    derivation_event_stream=True,
    exact_rule_ids=True,
    forward_backward_frontiers=True,
    native_geodesic_score=True,
    native_cost_vector=True,
    stamp_passthrough=True,
)
```

B.3 Backend selection

```
def choose_backend(
    requested: str,
    detected: KernelCapabilities,
) -> KernelControlPort:
    if requested == "native":
        if not detected.forward_backward_frontiers:
            raise UnsupportedKernelCapability("native geodesic control")
        return Patham9NativeGeodesicBackend(detected)
    if requested == "overlay":
        return Patham9OverlayBackend(detected)
    if requested == "legacy":
        return Patham9LegacyMicrostepBackend(detected)
    if detected.forward_backward_frontiers:
        return Patham9NativeGeodesicBackend(detected)
    return Patham9LegacyMicrostepBackend(detected)
```

C Property catalogue for implementation

Property	Generator	Assertion

Evidence idempotence	Random packet set E	`join(E,E)==E`.
Evidence commutativity	Random A, B	`join(A,B)==join(B,A)`.
Evidence associativity	Random A, B, C	Two parenthesizations equal.
Projection bounds	Random nonnegative counts, p_0, k	$0 \leq s, c \leq 1$.
Prior round trip	Random counts and fixed prior	Export(import(counts)) equals counts.
Stamp determinism	Random basis permutation	Same integer mapping.
Overlap safety	Shared basis ID	Binary stock application rejected.
Context separation	Same term, unequal contexts	No automatic aggregate.
Replay	Stored manifest	Result and trace digests equal.
Rollback	Random revision chain	Restored pointer equals selected ancestor.
Proof de-dup	Paths with same provenance key	One evidence valuation.
Independent proofs	Disjoint provenance keys	Joined valuation is additive.
Disjoint commutator	Declared independent actions	Defect below tolerance.
Budget monotonicity	Smaller budget	Cannot expand more nodes.
Cache soundness	Change one fingerprint field	Cache miss.
Promotion idempotence	Repeat same idempotence key	No second state change.

D Glossary

Term	Meaning
Atlas	Context-indexed family of local proof and probabilistic charts.
Chart	Temporary local probability model with explicit prior and assumptions.
Control mass	Search or bridge weight assigned to raw proof paths.
Evidence mass	Valuation of causally/provenance-distinct evidence units.
Evidence basis unit	Atomic unit used by the hard overlap and stamp policy.
Evidence packet	Context-indexed positive/negative empirical contribution with provenance.
Proof class	Evidence-aware component of syntactically distinct proof paths.
Rewrite cell	Explicit comparison or higher identification between parallel proof fragments.
Curvature probe	Difference between two locally reordered action composites.
Holonomy	Accumulated transport defect around a reversible proof/context loop.
Descent obstruction	Failure of local probability charts to admit a global gluing in a chosen class.
Reversible complement	Information retained so a visible update can be replayed or undone.
Kernel control port	Stable interface hiding legacy, overlay, and native geodesic patham9 control.
Promotion	Typed transition from ephemeral inference artifact to persistent state.

References

[1] patham9, *PLN: Modern PLN implementation for Hyperon*, GitHub repository, public main branch inspected July 12, 2026, https://github.com/patham9/PLN.

[2] patham9, *Deriver.metta*, public main branch inspected July 12, 2026, https://github.com/patham9/PLN/blob/main/src/Deriver.metta.

[3] patham9, *Formulas.metta*, public main branch inspected July 12, 2026, https://github.com/patham9/PLN/blob/main/src/Formulas.metta.

[4] patham9, *Rules.metta*, public main branch inspected July 12, 2026, https://github.com/patham9/PLN/blob/main/src/Rules.metta.

[5] patham9, *Translator.metta*, public main branch inspected July 12, 2026, https://github.com/patham9/PLN/blob/main/src/Translator.metta.

[6] patham9, *Repository build and test scripts*, public main branch inspected July 12, 2026, https://github.com/patham9/PLN.

[7] petta-memory, *Extending patham9's PLN Toward piPLN: Architecture, Concepts, Mathematics, Current Implementation, and Proposed Convergence*, technical design report, July 11, 2026.

[8] B. Goertzel, *πPLN: Paraconsistent PLN Without a Global Probability Space*, manuscript, May 27, 2026.

[9] B. Goertzel, *Geodesic Inference Control*, manuscript, December 5, 2025.

[10] B. Goertzel, *Paraconsistent Geodesic Modal HoTT, Semantic Elegant Normal Form, and TransWeave for Identity-Aware Commonsense Reasoning with PLN*, manuscript, December 15, 2025.

[11] B. Goertzel, *Causal Coding and the General Causal-Continual-Learning Theorem*, manuscript, December 1, 2025.

[12] B. Goertzel, *The Coupling Dilemma: Reach versus Conditioning, Sharing versus Forgetting, and Catastrophic Forgetting as Curvature of the Task Manifold*, manuscript, 2026.

[13] B. Goertzel, M. Ikle, I. F. Goertzel, and A. Heljakka, *Probabilistic Logic Networks: A Comprehensive Framework for Uncertain Inference*, Springer, 2008.

[14] C. H. Bennett, Logical reversibility of computation, *IBM Journal of Research and Development* 17 (1973), 525–532.

[15] R. L. Dykstra, An algorithm for restricted least squares regression, *Journal of the American Statistical Association* 78 (1983), 837–842.

[16] S. Amari and H. Nagaoka, *Methods of Information Geometry*, American Mathematical Society and Oxford University Press, 2000.